Python Programming - Day 01

Introduction to Python and Its Interpreter

Python is a high-level, general-purpose programming language known for its readability and simplicity. It uses an interpreter, which means Python code is executed line by line without requiring compilation before execution.

Why Python Uses an Interpreter

Python code is processed by the Python interpreter, which:

- Executes code directly, line by line
- Provides immediate feedback during development
- Allows for interactive programming through the Python shell
- Facilitates rapid development and debugging

Difference Between Interpreter and Compiler

Interpreter	Compiler
Translates and executes code line by line	Translates entire program at once before execution
No separate executable file is created	Creates a standalone executable file
Generally slower execution but faster development	Faster execution but longer initial processing time
Errors are reported one at a time as they're encountered	All errors are reported after compilation attempt
Memory efficient as only current line needs to be in memory	Typically requires more memory during compilation
Python, Ruby, JavaScript use interpreters	C, C++, Java use compilers

Comments in Python

Comments are non-executable text used to document code. They're ignored by the interpreter.

Types of Comments

```
# This is a single-line comment

'''
This is a multi-line comment
using triple single quotes
'''

"""
This is also a multi-line comment
using triple double quotes
"""
```

Uses of Comments

- Explain code functionality
- Document the purpose of functions and classes
- Temporarily disable code during testing
- Add reminders or TODOs for future development

Variables

Variables are names that reference values stored in memory.

Variable Declaration Examples

```
# Basic variable assignment
name = "John"
age = 25
is_student = True

# Multiple assignment
x, y, z = 10, "Hello", 3.14

# Value swapping
a, b = 5, 10
a, b = b, a  # Now a=10, b=5

# Incrementing variables
counter = 0
counter += 1  # counter is now 1
```

Keywords

Keywords are reserved words in Python that cannot be used as variable names:

```
False     class     finally   is        return
None      continue  for       lambda    try
True      def       from      nonlocal  while
and       del       global    not       with
as        elif      if        or        yield
assert    else      import    pass
break     except    in        raise
```

Data Types

Python has several built-in data types:

Numeric Types

```
# Integer
count = 10
print(type(count))  # <class 'int'>

# Float
pi = 3.14159
print(type(pi))  # <class 'float'>

# Complex
complex_num = 3 + 4j
print(type(complex_num))  # <class 'complex'>
```

Text Type

```
# String
name = "Python"
print(type(name))  # <class 'str'>

# String methods
print(name.upper())  # PYTHON
print(name.lower())  # python
print(len(name))  # 6
print(name[0])  # P
print(name[1:4])  # yth
```

Boolean Type

```
is_active = True
is_complete = False
print(type(is_active))  # <class 'bool'>
```

Sequence Types

```
# List (mutable)
fruits = ["apple", "banana", "cherry"]
print(type(fruits))  # <class 'list'>
fruits.append("orange")
print(fruits)  # ['apple', 'banana', 'cherry', 'orange']

# Tuple (immutable)
coordinates = (10, 20)
print(type(coordinates))  # <class 'tuple'>

# Range
numbers = range(5)  # 0, 1, 2, 3, 4
print(type(numbers))  # <class 'range'>
```

Mapping Type

```
# Dictionary
person = {
    "name": "Alice",
    "age": 30,
    "city": "New York"
}
print(type(person))  # <class 'dict'>
print(person["name"])  # Alice
person["email"] = "alice@example.com"
print(person)  # {'name': 'Alice', 'age': 30, 'city': 'New York',
'email': 'alice@example.com'}
```

Set Types

```
# Set (unordered collection with no duplicates)
colors = {"red", "green", "blue"}
print(type(colors))  # <class 'set'>
colors.add("yellow")
colors.add("red")  # Won't add duplicate
```

```
print(colors)  # {'green', 'yellow', 'blue', 'red'} (order may vary)

# Frozen Set (immutable set)
frozen_colors = frozenset(["red", "green", "blue"])
print(type(frozen_colors))  # <class 'frozenset'>
```

None Type

```
result = None
print(type(result))  # <class 'NoneType'>
```

Type Casting

Type casting is converting one data type to another.

```
# String to Integer
num_str = "123"
num_int = int(num_str)
print(num_int, type(num_int))  # 123 <class 'int'>

# Integer to String
num = 456
str_num = str(num)
print(str_num, type(str_num))  # 456 <class 'str'>

# String to Float
float_str = "3.14"
float_num = float(float_str)
print(float_num, type(float_num))  # 3.14 <class 'float'>

# Float to Integer (truncates decimal part)
pi = 3.14159
pi_int = int(pi)
print(pi_int, type(pi_int))  # 3 <class 'int'>

# Integer to Float
x = 5
x_float = float(x)
print(x_float, type(x_float))  # 5.0 <class 'float'>

# Converting to Boolean
print(bool(0))    # False
print(bool(1))    # True
```

```
print(bool(""))   # False
print(bool("Hi")) # True

# List, Tuple, and Set conversions
my_list = [1, 2, 3, 3]
print(tuple(my_list))  # (1, 2, 3, 3)
print(set(my_list))    # {1, 2, 3}
```

Introduction to Control Statements

Control statements alter the flow of program execution based on conditions or for repeated operations.

Types of Control Statements in Python:

1. **Conditional Statements**: Make decisions based on conditions
    - `if`, `elif`, `else`
2. **Loop Statements**: Execute code repeatedly
    - `for` loops
    - `while` loops
3. **Control Transfer Statements**: Change normal flow
    - `break`: Exit a loop prematurely
    - `continue`: Skip current iteration and proceed to next
    - `pass`: Null statement placeholder

These control statements will be covered in detail in subsequent lessons.